ChatGPT

Executive Summary

We propose a **real-time voice assistant demo** (~2 weeks scope) combining ASR, TTS, LangGraph orchestration, LiveKit streaming, Pipecat-like pipelines, and agent memory. For example, a *Voice QA Assistant* where users speak questions (via browser or phone), the system transcribes (ASR), processes with an LLM orchestration (LangGraph sub-agents, memory/RAG), and replies by synthesized speech (TTS) over the same stream. This uses **free-tier APIs** (or open-source) to minimize cost. Key components: a WebRTC layer (LiveKit) for low-latency audio, a speech-to-text API (e.g. Deepgram/AssemblyAI/Whisper), a LangGraph-based multi-agent orchestrator calling tools and memory, a vector DB (free Pinecone or Weaviate) for long-term memory, and a text-to-speech API (Google, Amazon, ElevenLabs, or open Coqui). We outline two prioritized project ideas, detailed architecture (with mermaid diagrams), memory design, recommended APIs (with free-tier limits), sample code, example GitHub projects, a 2-week plan (milestones/tests), deployment options, evaluation criteria, security/cost/privacy notes, and a README outline. All recommendations cite official docs or authoritative sources.

1. Project Ideas (1–2 week scope)

- **Voice Knowledge Assistant (Highest impact)**: A conversational assistant that answers user queries (e.g. FAQs or web knowledge) in real time. The user speaks a question; the system uses ASR (speech-to-text), feeds text to a LangGraph orchestrator that may call sub-agents or tools (e.g. a search or knowledge base via MCP), then replies via TTS. Prioritized because it showcases the full pipeline (ASR→LLM→TTS) plus memory for context and knowledge. This is feasible in ~2 weeks using free voice APIs and open models.
- **Voice Meeting Summarizer**: Record/transcribe a short meeting or dialog and allow voice queries about it. (User asks via voice, “What did we decide about X?” etc.) This uses ASR to transcribe, LLM to summarize/index, and memory (vector DB) for retrieval. More advanced (needs diarization/speaker ID), so lower priority, but shows off memory usage.
- **Voice Chatbot with Memory**: A simpler voice-chat companion that remembers user profile details (long-term memory) and maintains context. E.g., user can say “remember my favorite food is sushi,” and later ask “What do I like?”

2. Architecture (mermaid diagrams)

Below is a high-level pipeline and memory design. The **User** (browser/mobile) sends/receives audio via **LiveKit** (WebRTC). LiveKit streams audio to our backend, which runs the **ASR** engine (speech-to-text). The resulting text goes to a LangGraph **Orchestrator Agent**, which may call specialized **sub-agents** (as tools) and access memory. The orchestrator produces a text response, sent to the **TTS** service (text-to-speech), whose audio output is streamed back through LiveKit to the user. Short-term (session) memory is handled by LangGraph; long-term memory (facts, knowledge) is stored in a vector DB. Pipecat-like pipelines glue the streaming (STT/LLM/TTS) together.

```
flowchart TD
  subgraph UserSession
    User[User (Microphone/Speaker)]
  end
  subgraph LiveKitServer
    LK[LiveKit<br/>(WebRTC Transport)]
  end
  subgraph SpeechEngine
    ASR[ASR Service<br/>(free-tier STT API)]
  end
  subgraph Orchestration
    Orchestrator[LangGraph Orchestrator<br/>(LLM + Sub-agents)]
    SubAgent1[Expert Agent 1]
    SubAgent2[Expert Agent 2]
    MemoryDB[(Vector DB<br/>Long-term Memory)]
    STM[Short-term Memory<br/>(LangGraph context)]
  end
  subgraph Synthesis
    TTS[TTS Service<br/>(free-tier TTS API)]
  end

  User -->|audio in| LK
  LK -->|audio stream| ASR
  ASR -->|text| Orchestrator
  Orchestrator --> STM
  STM -.->|context| Orchestrator
  Orchestrator -->|tool calls| SubAgent1
  Orchestrator -->|tool calls| SubAgent2
  Orchestrator --> MemoryDB
  MemoryDB -.->|info| Orchestrator
  Orchestrator -->|response text| TTS
  TTS -->|audio out| LK
  LK -->|audio to user| User
```

```
flowchart LR
  Orchestrator --> STM[Short-term Memory (session context)]
  Orchestrator --> LTM[(Long-term Memory (Vector DB))]
  STM -.->|stores| LTM
  LLM -.->|embeddings→store| LTM
```

3. ASR and TTS APIs (free tiers and open-source fallback)

We list popular speech APIs, their free limits, and sample usage. When possible we cite official data.

ASR (Speech-to-Text):

- **Google Cloud Speech-to-Text:** 60 minutes free per month + \$300 credit【14†L79-L83】. High accuracy, supports 100+ languages. Requires GCP account and a Cloud Storage bucket. Example (Python):

```
from google.cloud import speech_v2
client = speech_v2.SpeechClient()
audio = speech_v2.RecognitionAudio(uri="gs://bucket/audio.flac")
config = speech_v2.RecognitionConfig(language_code="en-US")
response = client.recognize(config=config, audio=audio)
transcript = response.results[0].alternatives[0].transcript
```

- **IBM Watson STT:** 500 free minutes/month【27†L141-L144】. Widely used, supports customization. Requires IBM Cloud API key.
- **AWS Transcribe:** 60 free minutes/month for 12 months (standard tier) on AWS Free Tier. Good quality, requires S3.
- **AssemblyAI:** Provides **\$50 free credit** (hundreds of free hours at low volume)【2†L42-L49】. Easy REST API. Good for real-time. Sample (Python):

```
import requests
res = requests.post("https://api.assemblyai.com/v2/transcript",
    headers={"authorization": "YOUR_KEY"},
    json={"audio_url": "https://link/to/audio.mp3"})
text = res.json()["text"]
```

- **Deepgram:** \$200 free credit on signup (e.g. ~100 hours)【13†L1-L4】, real-time streaming supported with ~250ms latency【39†L156-L163】. Example (Python streaming):

```
from deepgram import Deepgram
dg = Deepgram("YOUR_DEEPGRAM_API_KEY")
response = await dg.transcription.prerecorded({"buffer": audio_bytes},
{"punctuate": True})
transcript = response["results"]["channels"][0]["alternatives"][0]
["transcript"]
```

- **OpenAI Whisper (via API):** No free tier; costs ~\$0.0067/sec. As open source, **Whisper local** is free (small/medium models run on CPU with latency ~1-2× real time)【39†L156-L163】. For a pure free

option, running Whisper (via whispercpp or HuggingFace inference) gives unlimited transcription (self-hosted costs CPU cycles).
- **Open-Source Engines:** Coqui STT (modern fork of Mozilla DeepSpeech) and Vosk provide free offline ASR. They require self-hosting but remove API limits (accuracy is lower).

TTS (Text-to-Speech):

- **Google Cloud Text-to-Speech:** 1 million chars free per month for WaveNet voices【37†L680-L688】. High-quality (DeepMind WaveNet/Neural2). Requires GCP key. Example (Python):

```
from google.cloud import texttospeech
client = texttospeech.TextToSpeechClient()
input_text = texttospeech.SynthesisInput(text="Hello world")
voice = texttospeech.VoiceSelectionParams(language_code="en-US",
ssml_gender=texttospeech.SsmlVoiceGender.NEUTRAL)
audio_config =
texttospeech.AudioConfig(audio_encoding=texttospeech.AudioEncoding.MP3)
res = client.synthesize_speech(input=input_text, voice=voice,
audio_config=audio_config)
open("output.mp3","wb").write(res.audio_content)
```

- **AWS Polly:** 5 million chars free for first 12 months (standard voices)【37†L680-L688】. Neural voices available beyond free-tier. Good quality, supports SSML.
- **Microsoft Azure TTS:** 500k chars free per month (neural voices)【37†L680-L688】. High-quality (WaveNet-like), multi-lingual.
- **ElevenLabs:** ~10k chars free/month【37†L680-L688】 (creative studio-quality voices). Limited free usage.
- **Coqui TTS (Open Source):** Free self-hosted TTS with decent quality (ESPN+LJ speech models). No API – you must run a server (model sizes ~400MB).
- **Other:** gTTS (Google Translate API, simple but lower quality), NaturalReader/Balabolka (desktop apps)【37†L680-L688】 for demos.

Summary: Google/Azure/AWS give substantial free quotas【37†L680-L688】. For fully free/self-hosted, use OpenAI Whisper+Coqui. Example integration: STT via Whisper locally, and TTS via Coqui, eliminating API costs (at expense of setup complexity).

4. Memory Design

Short-term (Session) Memory: Maintain conversation context per session/participant. LangGraph (and LangChain) support built-in chat buffers【31†L179-L187】. E.g., use `ConversationBufferMemory` or let LangGraph auto-track history. This keeps the last N turns for continuity. For example, LangChain:

```
from langchain.memory import ConversationBufferMemory
memory = ConversationBufferMemory()
```

Language-chain or LangGraph agents will automatically append user/assistant messages to this context.

Long-term Memory: Persistent knowledge (facts, documents, previous interactions). Use a **vector database** with embeddings. Good free choices:

- **Pinecone (Starter plan):** Free index (up to ~100k vectors of 1536 dims)【21†L65-L73】. Easy to use, supports upserts/queries.
- **Weaviate Cloud:** Free tier with limited data, or self-host docker.
- **Qdrant:** Free community version or small free cloud plan.
- **Chroma:** Open-source embed store (single file). Good for very small projects (no server).

Embed and store relevant content (e.g. key chat memory, scraped text, user profile) as vectors. Schema: each memory item has `{id, embedding, metadata: {text, timestamp, source, user_id,...}}`. Compute embeddings via a free model (e.g. HuggingFace all-MiniLM-v2 or OpenAI/Text-Embedding-3-small for moderate cost). Use dimensionality ~384 or 768 for miniLM. Example Pinecone usage:

```
import pinecone, uuid
pinecone.init(api_key="PINECONE_API_KEY", environment="us-west1-gcp-free")
idx = pinecone.Index("memories")
vec = embedder.encode("Sample memory text").tolist()
idx.upsert(vectors=[(str(uuid.uuid4()), vec, {"type": "conversation", "user":
"Alice"})])
```

Retrieval: on user query, embed query and run `idx.query(queries=[qvec], top_k=5)`.

Retention/Privacy: Only store sanitized, relevant info. Drop trivial chit-chat; only persist meaningful facts or logs. Use TTL policies: e.g., delete embeddings older than 30 days if not referenced. Ensure any personal data is hashed or deleted on user request (GDPR compliance).

Vector DB choice: Pinecone (free sandbox) is easiest for demo, with Python SDK. Weaviate or Qdrant are alternatives if Pinecone waitlisted (the Pinecone free plan may have a signup queue). For truly free, run Chroma/Qdrant locally (but demoing in 2 weeks probably fine with Pinecone).

5. LangGraph Integration (Orchestrator + Sub-agents + MCP)

LangGraph (LangChain) lets a *main agent* orchestrate sub-agents as tools【31†L179-L187】. We adopt the **subagents** pattern: the main "supervisor" agent keeps overall context and invokes specialized sub-agents (via tools) for tasks. For example, a research sub-agent can fetch info from a database or web, a calculator sub-agent can compute, etc. This prevents context bloat in the main agent【31†L130-L139】.

Example Pattern:

```
from langchain.agents import create_agent
from langchain.tools import tool

# Define a sub-agent (stateless)
research_agent = create_agent(model="openai: gpt-3.5-turbo", tools=[...])
@tool("research", description="Search the knowledge base")
def call_research_agent(query: str) -> str:
    result = research_agent.invoke({"messages":
[{"role":"user","content":query}]})
    return result["messages"][-1].content

# Main orchestrator agent uses the sub-agent as a tool
orchestrator = create_agent(model="openai: gpt-4o", tools=[call_research_agent])
```

Here `orchestrator.invoke()` routes queries: if it uses `call_research_agent`, it triggers the sub-agent【31†L179-L187】. See LangChain's Subagents docs for details.

MCP Tool Calls: LangChain supports calling MCP (Model Context Protocol) servers as tools. Using the MCP Python SDK, you can call a tool on a server by name【35†L429-L445】:

```
from mcp import Client
async with Client("http://localhost:8000") as client:
    res = await client.call_tool("weather", {"location": "Paris"})
    answer = res.structured_content["result"]
```

In LangGraph, you'd typically wrap MCP calls in a `@tool` function, so the LLM can invoke it as needed. MCP provides standardized access to external data (file system, DB, custom APIs) with JSON schemas【35†L429-L445】.

LangGraph Example (Orchestrator + Tool):

```
from langchain.agents import create_agent
from langchain.tools import tool

# Sub-agent as tool (calls an LLM)
math_agent = create_agent(model="google_genai:gemini-3.5-flash", tools=[])
@tool("calculate", description="Perform arithmetic calculation")
def call_math_agent(expr: str) -> str:
    return math_agent.invoke({"messages":[{"role":"user","content":expr}]})
["messages"][-1].content

main_agent = create_agent(model="google_genai:gemini-3.5-flash",
tools=[call_math_agent])
```

(Code from LangChain docs【31†L179-L187】, with generic LLM placeholders.) The main agent can now say "@calculate("2+2")" to call the sub-agent.

LangGraph vs LangChain Agents: We run these in Python (no GPUs needed, just API keys). The main orchestrator agent could be either a single chain or a LangGraph graph of nodes (if multi-step planning needed) – for a demo, a single agent with tool calls suffices.

6. LiveKit Integration (Real-time Streaming)

LiveKit is an open-source WebRTC platform for live audio/video. We use LiveKit to stream audio between the client and our server with minimal latency (tens of milliseconds)【39†L129-L137】. Instead of HTTP request/response per query, LiveKit maintains a persistent bidirectional connection: as the user speaks, audio flows continuously in, and the agent's replies stream out. This "listen-think-speak" pipeline cuts round-trip latency to ~100–300 ms【39†L129-L137】.

LiveKit provides **Python and Node.js SDKs**. The LiveKit Agents framework simplifies building voice agents by handling the WebRTC layer and streaming integration. For example, in Python:

```
from livekit.agents import AgentSession, TurnHandlingOptions, MultilingualModel
session = AgentSession(
    stt="deepgram/nova-3",      # choose STT model
    llm="openai/gpt-4o",        # LLM model
    tts="cartesia/sonic-3",     # TTS model
    turn_handling=TurnHandlingOptions(turn_detection=MultilingualModel())
)
session.run()  # joins LiveKit room and starts audio pipeline
```

(See LiveKit quickstart【20†L64-L72】; snippet above simplified.) This sets up the full pipeline: LiveKit captures user audio, sends to STT, feeds text to LLM, then TTS for response. The Python server runs an **AgentServer** that schedules AgentSessions per user. The LiveKit tutorial confirms this STT→LLM→TTS pipeline as standard【39†L73-L81】.

Latency/Quality: LiveKit docs list STT/TTS models with latencies. For example, Deepgram Nova-3 (~250 ms latency, excellent accuracy) and AssemblyAI streaming (~300 ms) are high-accuracy STT options【39†L156-L163】. For TTS, Cartesia Sonic-3 (~90 ms, high quality) or ElevenLabs (~150 ms) are top performers【39†L172-L179】.
Deployment: We can run LiveKit self-hosted (requires Go server) or use LiveKit Cloud (free trial tier). For a 2-week demo, running a local LiveKit server in Docker is sufficient (no paid infra needed). Use the Python SDK (`livekit-agents`) via `pip install livekit-agents`.

7. Pipecat (Pipeline Orchestration)

Pipecat is an open-source voice agent framework that combines many of the above pieces【11†L374-L382】. It provides CLI scaffolding (`pipecat create quickstart`) and a pipeline model where each

“frame” of audio is processed by chained components. Pipecat’s key features (voice-first, pluggable pipelines, multi-agent) match our needs【11†L349-L358】.

A Pipecat quickstart (Python) might use Deepgram STT, an LLM (OpenAI), and Cartesia TTS: the CLI scaffold runs a live streaming bot. Example steps:

```
uv tool install "pipecat-ai[cli]"   # install Pipecat CLI
pipecat create quickstart           # scaffold a bot (uses Deepgram, OpenAI,
Cartesia by default)
cd pipecat-quickstart
uv run pipecat dev                 # run locally
```

(From Pipecat docs【25†L208-L217】.) You would then edit `pipeline.py` or config to plug in chosen STT/TTS and agent logic.

Pipecat allows defining multiple pipelines (agents) and flows, including fallback, handoff, and multi-agent setups【11†L377-L382】. It’s production-grade (used by AI companies) but also suits demos: everything is local/Python. However, Pipecat’s default examples use paid APIs (Deepgram, OpenAI, Cartesia). For free-tier adaptation, swap in Whisper/Coqui and HuggingFace LLMs if needed.

In essence, **Pipecat or LiveKit Agents** can manage the real-time audio transport and pipeline. In our architecture, we could use Pipecat pipelines instead of raw AgentSession: either approach achieves similar flow. (We won’t use both; pick one for implementation.) For this plan, we emphasize LiveKit Agents (above) and mention Pipecat as an alternative orchestration framework with similar goals【11†L374-L382】.

8. Example GitHub Projects (clones/adapt)

- **LiveKit Agents Framework** (python) – Official LiveKit repo. Shows how to build realtime voice agents with STT/LLM/TTS and even telco integration【49†L325-L333】【49†L338-L346】. Good for reference (and code example).
- **Pipecat** – The Pipecat repo (GitHub: pipecat-ai/pipecat) itself【11†L349-L358】. Contains examples of multi-agent pipelines, flows, and a quickstart voice bot. Useful for flow design.
- **LangGraph Chatbot (extrawest)** – extrawest/fastapi-langgraph-chatbot-with-vector-store-memory-mcp-tools-and-voice-mode. This showcases many features we need: LangGraph orchestration, Qdrant vector memory, cross-chat memory, MCP tools, and **LiveKit-based voice mode (Deepgram STT, Cartesia TTS, Silero VAD)**【44†L330-L339】【44†L301-L309】. A very relevant reference for architecture (uses FastAPI + React frontend).
- **LangGraph Voice Call Agent** – ahmad2b/langgraph-voice-call-agent. Demonstrates hooking a LangGraph agent into LiveKit for full-duplex voice【47†L304-L312】【47†L316-L323】. It includes VAD, Deepgram STT/TTS, and turn detection. Great for seeing how LangGraph + LiveKit glue together.
- **FreeCodeCamp Multi-Agent Example** – sandeepmb/freecodecamp-multi-agent-ai-system. Companion code for the LangGraph+MCP multi-agent tutorial【23†L39-L48】. Shows LangGraph workflows with MCP tool servers. Not voice-specific, but good for LangGraph+MCP patterns.

- **LiveKit TypeScript Agent Example** – In LiveKit Agents repo under `examples/`, there are demos (chatbot, Python & TypeScript). Inspect `examples/` in livekit/agents for reference pipelines (e.g. VoiceRepeater example).

Each repo provides architectures and code snippets. We can clone or adapt components (e.g. copy voice pipeline code from **langgraph-voice-call-agent** or memory code from **extrawest** repo).

9. 2-Week Implementation Plan

Week 1: Foundation and Core Pipeline

- *Day 1:* **Setup** – Define project scope; signup for free API keys (Google/Azure/AWS/Deepgram/AssemblyAI as needed). Install tools (Python 3.11+, `uv` or venv). Clone example repos for reference (e.g. langgraph-voice-call-agent).
- *Day 2:* **ASR Integration** – Implement speech transcription. Try e.g. AssemblyAI or Deepgram API calls on test audio, plus Whisper offline fallback. Write code to stream or upload audio and get transcript. Test with sample audio files.
- *Day 3:* **TTS Integration** – Implement text-to-speech. Try Google Cloud TTS and an open alternative (Coqui or ElevenLabs free-tier). Write code to convert text to audio (MP3/PCM) and play. Test with simple phrases.
- *Day 4:* **LangGraph Orchestrator Skeleton** – Create a LangGraph (LangChain) agent or graph that takes text input and returns text output (using a free or cheap LLM, e.g. OpenAI GPT-3.5 turbo with free trial credit). Test basic prompt/response. Sketch sub-agent tools (MCP, knowledge lookup).
- *Day 5:* **End-to-End Pipeline (Without LiveKit)** – Combine ASR + LLM + TTS in sequence. E.g., feed prerecorded audio → transcript → LangGraph agent → response text → TTS → save audio. Verify with local testing (no streaming). Write unit tests for each component.
- *Milestone:* Working proof-of-concept: "STT → LLM → TTS" pipeline works on test data.

Week 2: Orchestration, Memory, Streaming, Deployment

- *Day 6:* **LangGraph Multi-Agent Logic** – Enhance the LLM workflow. Add memory integration: on each turn, store user & assistant messages in short-term memory (LangGraph context) and optionally long-term memory via vector store. If implementing retrieval (RAG), connect vector DB: on each user query, query semantic DB for relevant facts. Test with manual inserts.
- *Day 7:* **MCP Tools & Skills** – Add tool calls. Example: a calculator tool via MCP, a web-search tool, or call a sample MCP server (like file-reader, notes DB). Ensure LangGraph can call these using MCP client. Test by querying knowledge (e.g., "Calculate 12*37.") and check tool usage.
- *Day 8:* **LiveKit Streaming Integration** – Integrate LiveKit for real-time audio. Start a local LiveKit server (e.g. `docker-compose up livekit`). Use LiveKit's Python SDK (`livekit-agents`). Implement an `AgentSession` with chosen STT/TTS/LLM. Test with a browser demo (LiveKit Web SDK) or local script to speak via mic. Debug latency and audio quality.
- *Day 9:* **Frontend/UX** – Build a simple UI (React or even pure HTML) with microphone access, using LiveKit JS SDK. Show incoming agent audio (speakers). Optionally, add text chat fallback. Ensure it works in-browser. Test end-to-end conversation via voice.
- *Day 10:* **Testing & Polish** – Systematically test. Create automated tests: feed prerecorded WAVs and ensure responses. Check memory recall (e.g. "remember X" scenarios). Refine prompts and error handling.

- *Milestone:* Voice assistant prototype working live.

Deliverables: Code repo, README, working demo (locally or deployed), test scripts, design diagrams.

10. Deployment (Free Hosting/CI)

- **Backend:** Containerize server (FastAPI or similar). Use **Render.com** free tier (750 h/month) or **Fly.io** free (~3 shared-cpu-1x). Alternatively, use **Heroku** with free buildpacks (limited credits) or **Deta.sh** (Hobby). Secure env vars for API keys.
- **LiveKit Server:** Can run in same container (if Fly.io) or separately on free server. If using LiveKit Cloud free trial, skip self-host.
- **Frontend:** Host static UI on **Vercel** or **GitHub Pages** (if React, Vercel). Connect to backend via WebRTC (no CORS issues).
- **CI/CD:** Use GitHub Actions. Example: on push to `main`, build & push Docker image to Render/Fly. Or use Vercel auto-deploy.
- **Vector DB:** Use Pinecone's free cloud (no need for self-host). Keep credentials in env.
- **Secrets:** Store API keys in deployment secrets (Render/Deta env vars).

11. Evaluation & Demo Script

Checklist:

- **Audio Pipeline:** System transcribes spoken input with <10% WER (Word Error Rate) for clear speech; responds via TTS.
- **Latency:** Conversation feels responsive (peak round-trip <1s). LiveKit streaming works (no audio lag).
- **Memory:** On multi-turn conversation, the agent recalls earlier details ("What's my favorite color?" after user said it).
- **Context Switching:** Agent handles follow-ups ("and what about X?" referencing previous answer).
- **Tools:** If integrated, test at least one tool (calculator or search).
- **Quality:** LLM answers correctly or gracefully handles unknowns. TTS voice is intelligible.
- **Reliability:** App doesn't crash on silence or gibberish; idle timeouts handled.

Demo Script: (for interview or showcase) 1. **Intro:** "Hello, can you hear me?" (Should respond by voice).
2. **Basic Q&A:** "What's the weather in Paris?" (If connected to a weather API via MCP).
3. **Memory Test:** "Remember that I like pizza." ... later, "What do I like?" (Agent should say pizza).
4. **Complex Task:** "Calculate 13 times 19." (Should call calculator tool).
5. **Streaming Demo:** Chat like a natural conversation, showing smooth STT->agent->TTS turn-taking.
6. **Multi-modal (if done):** Show an image (if integrated) and ask about it.

Record responses and time for each. Show frontend reacting (message transcript + audio playback).

12. Security, Cost, Privacy

- **Security:** Use HTTPS for all API calls. Protect API keys (backend only; do not expose in frontend). Use authentication if deploying publicly. Limit CORS. Regularly rotate keys.

- **Privacy:** Only record/transcribe audio with user consent. Delete or encrypt stored transcripts. In memory, avoid storing sensitive PII. For any logged data, strip user identifiers.
- **Cost:** All chosen services have free tiers. Major costs: minor compute (RPS/duration), minimal network. Track API usage: limit to demo usage. For example, 1 hr of audio at Deepgram = \$0 (on credit). Pinecone free covers demo data. ElevenLabs free (10k chars ~2 pages) is tiny. Overall cost ~\$0 beyond free limits for a short demo.
- **Performance:** On free tiers, watch rate limits (e.g. Pinecone: 100K vectors, 60QPS). For voice, WebRTC uses a small bandwidth per user (kbps).
- **Scaling:** This is a demo, so scale modest. Use single processes; parallel LiveKit sessions possible on one server.

13. README & Portfolio Write-up

README Structure:

- **Title/Description:** "Voice AI Agent (ASR+TTS+Memory+LangGraph+LiveKit)"
- **Demo/Overview:** Short explainer and embedded diagram (mermaid).
- **Features:** Bulleted: real-time STT→LLM→TTS; persistent memory; multi-agent orchestration; web interface.
- **Architecture:** Describe components (LiveKit, LangGraph, memory DB) with diagram. Cite key libraries.
- **Setup:** Requirements (Python3.11+, API keys), installation steps (pip, environment), configuration (API keys in `.env`), run instructions.
- **Usage:** How to start server + frontend, how to use voice interface.
- **Testing:** How to run tests, example commands for demo script.
- **Next Steps:** Extensions (multimodal, more agents).
- **Credits/Sources:** Link to docs (LiveKit, LangChain, Pipecat) and libraries used.

Portfolio Bullets:

- "Developed a **real-time voice AI assistant** integrating streaming ASR, LLM orchestration, and TTS using LiveKit and LangChain's LangGraph."
- "Implemented multi-agent workflow with **short- and long-term memory** (vector DB) for context-aware conversation."
- "Utilized **free-tier speech APIs** (Deepgram/AssemblyAI and Google/AWS TTS) and fallback open-source models (Whisper, Coqui)【39†L156-L163】【37†L680-L688】."
- "Deployed on free cloud (e.g. Render, Vercel) with CI; the system runs entirely without paid GPUs."
- "Demo-ready architecture with Mermaid diagrams and scripted evaluation checklist for interviews."

Executive Summary (Opening):
This project is a **full-stack voice assistant** demo built with open-source frameworks and free-tier services. It demonstrates how to combine real-time audio streaming (LiveKit) with AI (ASR, LLM, TTS) and agent orchestration. The design prioritizes responsiveness and memory: the assistant listens (STT), reasons over user data and tools (LangGraph with memory), and speaks (TTS) in a continuous loop【39†L73-L81】【39†L129-L137】. All components—ASR, TTS, memory, orchestration—are sourced from free tiers or open models (Google/AWS/AssemblyAI for ASR/TTS【37†L680-L688】【14†L79-L83】, Pinecone for memory【21†L65-L73】, LangChain/LiveKit for orchestration). We include sample code (Python) for each part,

architecture diagrams, and a detailed 2-week plan. This rigorous report serves as both a blueprint and tutorial for building advanced voice AI without requiring GPU or heavy infrastructure.
